



Design of a Lightweight Fast Fourier Transformation for FALCON using Hardware-Software Co-Design

Suraj Mandal, Debapriya Basu Roy
Indian Institute of Technology Kanpur
Kanpur, India
{surajmandal,dbroy}@cse.iitk.ac.in

ABSTRACT

Lattice-based post-quantum cryptographic algorithm FALCON needs to execute the time-critical Fast-Fourier Transformation (FFT). Existing works in the literature have explored hardware for FFT of FALCON using Cooley-Tukey. In this work, we have designed an efficient hardware-software co-design of FFT for FALCON using Winograd's FFT method. Winograd's FFT is a widely adopted technique for FFT and reduces the multiplication counts for higher-radix FFT than the Cooley-Tukey, with a penalty of some extra addition/subtraction. Our Winograd radix-8 framework for FFT outperforms the traditional Cooley-Tukey method. Moreover, our proposed architecture is flexible in adopting different instruction sets and can also be configured for any type of FFT method with specific instruction sets.

CCS CONCEPTS

- **Security and privacy** → **Hardware security implementation**;
- **Efficient FFT Techniques** → *Cooley-Tukey, Winograd*.

KEYWORDS

FALCON, Post-Quantum Cryptography, Fast Fourier Transformation (FFT), Winograd's FFT

ACM Reference Format:

Suraj Mandal, Debapriya Basu Roy. 2024. Design of a Lightweight Fast Fourier Transformation for FALCON using Hardware-Software Co-Design. In *Great Lakes Symposium on VLSI 2024 (GLSVLSI '24)*, June 12–14, 2024, Clearwater, FL, USA. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3649476.3660370>

1 INTRODUCTION

The advancement of quantum computers has boosted the need for post-quantum cryptographic algorithms. In 2022, NIST has announced four post-quantum cryptographic algorithms to be standardized. Among these finalists, FALCON is one of the digital signature algorithms and is acknowledged for its fast signature verification. But, due to the involvement of Fast-Fourier Transformation (FFT) in the algorithm, the software implementation of FALCON results in high latency. FALCON has a polynomial degree of 512 for security level I and 1024 for security level V. However, while

executing the entire algorithm, FALCON needs to perform FFT of polynomials starting from degree two to 512 in case of security level I and 1024 in case of security level V. Due to the algorithmic complexity, few works in the literature have explored efficient implementation of FALCON [2, 3, 10, 11]. The only work that tried to implement area and power efficient FFT module for FALCON is [2]. Implementing FFT for FALCON is challenging because it requires double-precision floating-point arithmetic, compliant with IEEE 754 standards. This essentially means that the FFT implementation would require 64 bit architecture for floating point adder, multiplier, and subtractor. This distinguishes the FFT implementation of FALCON with the NTT implementation of other post-quantum secure algorithms where the adder, multipliers, and subtractor are based on modular arithmetic with only 24 bit modulus (at most). Therefore, the implementation of NTT in those lightweight PQC algorithms can be efficiently pipelined, resulting in a fast and compact design [12, 13]. However, being based on 64 bits double-precision floating-point arithmetic, the FFT implementation of FALCON is significantly more computationally intensive, which in turn increases the design overhead significantly for pipelined implementation.

To address this issue, in this paper we propose a lightweight design strategy for FFT implementation in FALCON using hardware/software codesign methodology. The main feature of our implementation are as follows:

- We have used the Winograd radix-8 butterfly unit (BFU) instead of the more popular Cooley-Tukey and Gentleman-Sande butterfly units for implementing FFT. The usage of Winograd is inspired by the requirement of the lower number of multipliers compared to its higher radix Cooley-Tukey variant. Additionally, the deployment of radix-8 BFUs would help us to reduce the loop iterations for computing FFT.
- Implementing Winograd's BFU requires the computation of complex adder, subtractor, and multiplier. Rather than developing a dedicated Winograd BFU, our hardware implementation includes designs of a complex multiplier, a complex adder, and a complex subtractor only. Our developed reconfigurable FFT processor can be used to implement any type of BFUs for FFT computation with its own instruction set architecture. In this work, we have focused on Winograd's BFU by invoking the hardware implementation of the developed complex multiplier, adder, and subtractor.

By following this approach, we have reduced the overhead of the FALCON's FFT processor significantly. Additionally, this approach gives us flexibility, allowing the designer to opt for different butterfly structures if required. For example, a Cooley-Tukey BFU can also be developed using our proposed processor by instantiating



This work is licensed under a Creative Commons Attribution International 4.0 License.

GLSVLSI '24, June 12–14, 2024, Clearwater, FL, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0605-9/24/06
<https://doi.org/10.1145/3649476.3660370>

appropriate instructions for complex adder, multiplier and subtractor. Additionally, we can also easily integrate FFT of varying length (an essential component for FALCON) in the proposed framework.

The rest of the paper is organized as follows. In Section 2, we have provided a brief description on FFT of FALCON and Winograd's BFU. Section 3 explores the existing implementations of FALCON. In Section 4, we have provided a detailed description of our proposed architecture. Experimental results have been discussed in Section 5. We conclude the paper in Section 6.

2 BACKGROUND

2.1 FALCON

The algorithm of FALCON involves key generation, signing and verification. Among these stages, FFT computation is needed in key generation and signing. FFT is computed recursively in the Key-generation for the LDL^* function. 26% of the total latency in keygen and 48% of the total latency in signing involves FFT computation [10]. So, acceleration of the FFT module will make the entire algorithm faster. Also, FALCON operates on floating point arithmetic. So, it is necessary to develop efficient floating point arithmetic modules.

2.2 FFT in FALCON

Fast Fourier Transformation (FFT) is the most time-consuming operation in the post-quantum cryptographic algorithm FALCON that operates in a ring $(x^n + 1)$. The equation of FFT transformation in of a N degree polynomial $A(x) = \sum_{i=0}^{n-1} a_i x^i$ for FALCON can be defined by $FFT_{A(x)} = \sum_{i=0}^{n-1} \gamma^i \cdot \omega_n^{ij} \cdot A_i$ where powers of γ and ω are complex roots of unity defined over $x^n + 1$ and are called as twiddle factors. The most well-known method for the computation of FFT is the radix-2 Cooley-Tukey algorithm [4]. The basic building block of the radix-2 Cooley-Tukey FFT algorithm is Cooley-Tukey radix-2 butterfly unit that consists of a complex number multiplier, a complex number adder and a complex number subtractor.

Algorithm 1 Radix-8 Winograd BFU Algorithm

- 1: **Input:** ω, a_i where, $i = 0, 1, 2, \dots, 7$
 - 2: **Output:** $\tilde{a}_i$ where, $i = 0, 1, 2, \dots, 7$
 - 3: $\omega_0 = \omega_1 = \omega_2 = \omega_4 = 1, \omega_3 = \omega_5 = -j, \omega_6 = -0.707106, \omega_7 = 0.707106$
 - 4: $s_1, s_5 = H_f(a_1, a_5), s_3, s_7 = H_f(a_3, a_7), s_0, s_4 = H_f(a_0, a_4), s_2, s_6 = H_f(a_2, a_6)$
 - 5: $s_8, s_9 = H_f(s_1, s_3), s_{10}, s_{11} = H_f(s_5, s_7)$
 - 6: $t_0 = s_0, t_1 = s_4, t_2 = s_2, t_3 = CMUL(s_6, \omega_3), t_4 = s_8, t_5 = CMUL(s_9, \omega_5), t_6 = CMUL(s_{10}, \omega_6), t_7 = CMUL(s_{11}, \omega_7)$
 - 7: $s_{12}, s_{13} = H_f(s_6, t_7), s_{14}, s_{16} = H_f(t_0, t_2), s_{15}, s_{17} = H_f(t_1, t_3)$
 - 8: $\tilde{a}_0, \tilde{a}_4 = H_f(s_{14}, t_4), \tilde{a}_1, \tilde{a}_5 = H_f(s_{15}, s_{12}), \tilde{a}_2, \tilde{a}_6 = H_f(s_{16}, t_5), \tilde{a}_3, \tilde{a}_7 = H_f(s_{17}, s_{13})$
 - 9: **return** $\tilde{a}_i$ where, $i = 0, 1, 2, \dots, 7$
-

2.3 Winograd's BFU

Winograd's FFT is another method for computing FFT and is known for the reduced number of multiplications than the Cooley-Tukey method while computing to higher-radix FFT [1, 7, 16, 18]. In [1], the authors have derived Winograd's DFT (Discrete Fourier Transformation) equations for polynomials of different degrees. In [16], the authors have tried to construct Winograd's butterfly units for the power of two points for computing DFT. We have constructed a similar radix-8 butterfly unit for FFT of FALCON security level I. The algorithm for the radix-8 BFU construction using Winograd's method has been provided in Algorithm 1. H_f in Algorithm 1 refers to the Hadamard gate expressed as the matrix $\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$ i.e. Hadamard gate consists of one complex number addition and one complex number subtraction. Similarly, the function $CMUL$ consists of the complex number multiplication where one of the inputs are coming from the twiddle factors and another input is coming from the Hadamard gate outputs. Please note that the total number of multiplications needed inside the radix-8 Winograd's BFU is only two, as the twiddle factors $\omega_0 = \omega_1 = \omega_2 = \omega_4 = 1, \omega_3 = \omega_5 = -j, \omega_6 = -0.707106$ and $\omega_7 = 0.707106$. An architectural diagram of our radix-8 Winograd's butterfly unit has been shown in Figure 1. The 'H' gate in Figure 1 refers to the H_f function in Algorithm 1. The

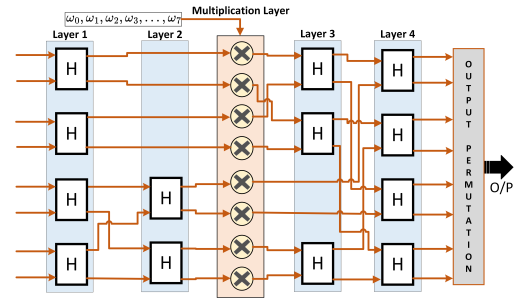


Figure 1: Winograd's Radix-8 Butterfly unit

work [8], shows that a radix- R Number Theoretic Transformation (NTT) will involve $R - 1$ multiplications outside the radix- R butterfly unit. As the method of computing NTT and FFT are similar, to compute a radix-8 FFT, we have adopted a similar technique. Thus, our radix-8 FFT butterfly unit involves 9 multiplications in total, whereas in radix-8 Cooley-Tukey butterfly unit, we need to compute 12 multiplications in total.

3 RELATED WORK

In this section, we shall discuss the existing works that have tried to accelerate the computation of FFT in FALCON. Due to the algorithmic complexity, there are a few that com In works that have tried to provide acceleration of FFT operations. In [10], the authors have accelerated the FFT/NTT computation of FALCON by using the NEON engine for an ARMv8 processor, and the authors have achieved a significant performance improvement for key generation/signing/verification of FALCON. Later on in [3], the authors proposed an accelerator for FALCON, where they have implemented the Keccak hash and NTT polynomial multiplication unit

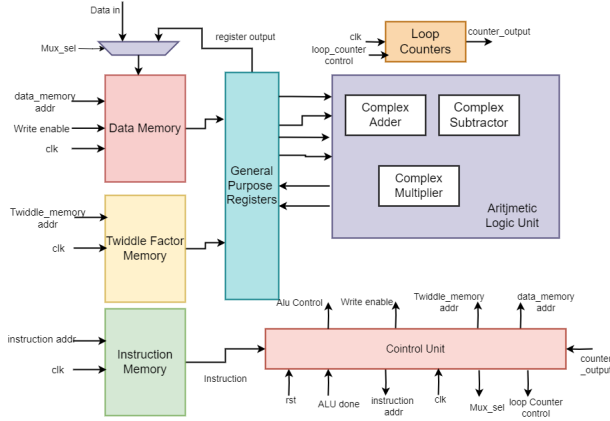


Figure 2: The proposed FFT processor framework

in hardware. The authors in [2], proposed an accelerated software implementation of FFT for FALCON on Cortex-A72 that achieves a significant improvement in performance compared to the previous works. The work [9] has implemented a hardware-software co-design for FALCON, where the discrete Gaussian sampler has been implemented on FPGA. The authors in [17] have implemented the key generation of FALCON using High-Level Synthesis and have tested their implementation results on Zynq Ultrascale+ FPGA board.

Winograd's FFT also has been explored in few of the works. The work [6] implemented a FFT processor using Winograd's technique. Later on in [5], the authors have implemented radix-16 based FFT using their approximated double precision number system. Although the work achieves a significant improvement over area overhead, there was a negligible improvement in latency cycles needed for computation.

4 PROPOSED WORK

4.1 The FFT Processor

This section will illustrate our proposed framework for the FFT processor. The block diagram of the developed FFT processor is shown in Fig. 2. The architecture consists of three types of memory: *data memory*, *instruction memory* and *twiddle factor memory*. Each cell of the *data memory* contains a single 64 bit double precision floating point value. Therefore, a complex number requires two cells in the data memory for its storage. The *twiddle factor memory* is a ROM that has a hardcoded value of all the twiddle factors. The instruction memory contains the instructions that are used to compute the FFT operations. The *general purpose registers* act as an interface between the Arithmetic logic unit (ALU) and acts as storage for temporary variables. The ALU contains hardware implementation of a complex number adder, multiplier and subtractor. The loop counters are used to implement iterative operations that are required for FFT computation. The entire design is controlled by a finite state machine based *control unit*. Next, we will describe our proposed hardware for double-precision floating point arithmetic

4.2 Double Precision Floating Point Arithmetic

A IEEE-754 double-precision floating-point multiplier has an input and output size of 64 bit with 1 bit for sign, 11 bits for exponent and 52 bits for mantissa, as shown in Figure 3. While performing



Figure 3: IEEE-754 Double Precision Format

floating point operations in FALCON, special cases like subnormal value, infinite or NaN do not occur [15]. So, we have used a generic 64 bit floating point multiplier without any exception cases [14]. Multiplication is performed by appending one bit extra to the mantissa for normalization purposes, as shown in Figure 4. One floating point double precision multiplier (FPMUL) consumes 9 DSPs to execute the multiplication in one clock cycle. The overhead of the FPMUL has been provided in Table 1.

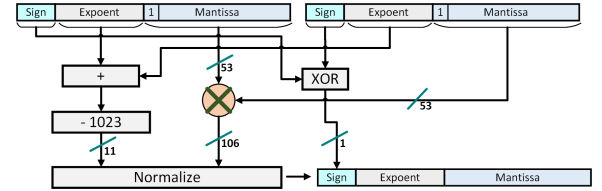


Figure 4: Double Precision Floating Point Multiplier

For double precision floating point adder/subtractor also, we have used generic double precision floating point adder/subtractor (FPADD/FPSUB) without any exception cases [14]. Our unified adder/subtractor has been shown in Figure 5. Depending upon the select signal *ADD/SUB* we can choose the operation to be performed, which will be either addition or subtraction. The result after subtraction could be very small. So, we need to normalize the subtraction result in such cases, i.e. left shift of the leading zeros after the decimal point. For normalization of the results in constant time, we have used a priority encoder that will decide how many bits should be shifted according to the two overflow bits after subtraction. For addition operation, we do not need the normalizer, as there should be only two cases:

- The addition of the mantissas may result in a carry overflow bit. We have right-shifted the mantissa by one after addition and increased the exponent by 1 in such cases.
- In case of no carry overflow, the result will be same as it is.

The overhead of the FPADD/FPSUB module has been reported in Table 1.

4.3 Complex Multiplier, Adder and Subtractor

For FFT, we have used the complex number addition/subtraction. A complex number addition/subtraction of two numbers $(a + i \cdot b)$ and $(c + i \cdot d)$ can be defined by $(a \pm c) + i \cdot (b \pm d)$. So, we need to perform two floating point adders/subtractors for the addition/subtraction operation of one pair of complex coefficients. Our complex number

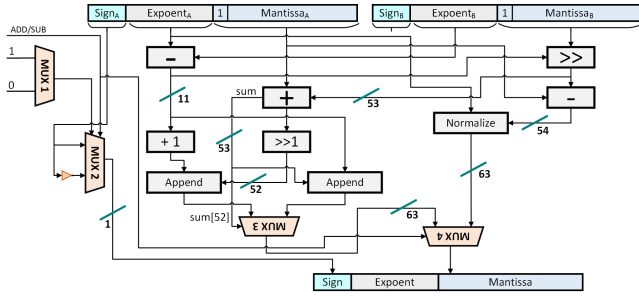


Figure 5: Double Precision Floating Point Adder/Subtractor

	LUT	FF	DSP
Floating Point Adder/Subtractor	1734	-	-
Floating point Multiplier	164	64	9
Complex Number Adder/Subtractor	2071	512	-
Complex Number Multiplier	3873	895	9

Table 1: Overhead of Floating point adder or subtractor/Floating point multiplier/Complex number adder or subtractor/Complex number multiplier in ZCU104+ FPGA board

adder/subtractor uses one floating point addition/subtraction module and computes the result in two clock cycles with an overhead of 2071 LUTs as shown in Table 1.

The multiplication of two complex numbers $(a + i \cdot b)$ and $(c + i \cdot d)$ is defined as $(ac - bd) + i \cdot (ad + bc)$. One such complex multiplier includes four floating point multiplications involving 9 DSPs to complete the multiplication of 53-bit mantissa in four clock cycles, one floating point double precision adder and one floating point double precision subtractor. Our complex number multiplier consumes 9 DSPs with a latency of 5. The overhead of the complex number multiplier has been shown in Table 1. The architecture of our complex multiplier has been shown in Figure 6 where A and B are two complex numbers and FPMUL, FPADD, FPSUB indicate floating point multiplier, adder and subtractor, respectively. Real and imaginary parts have been indicated by *re* and *im*, respectively.

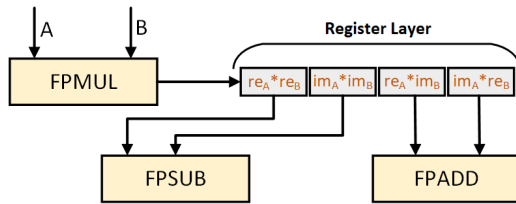


Figure 6: Complex Number Multiplier

4.4 Instruction Set Architecture

The different instructions that we have used for executing FFT is shown in Table 2. The *LOAD* instruction loads data into data memory (either from the external world or from general purpose registers). The *READ* instructions are used to upload data from the

Table 2: Instruction List

Instructions	Operations
LOAD_EXT DM_addr	External data gets stored in the addr DM_addr of data memory
READ_EXT DM_addr	Value at DM_addr of data memory is read to the external world
CM_ADD R_a, R_b, R_c	Performs complex addition between two operands (stored at R_a, R_b) and store the result in R_c
CM_SUB R_a, R_b, R_c	Performs complex subtraction between two operands (stored at R_a, R_b) and store the result in R_c
CM_MUL R_a, R_b, R_c	Performs complex multiplication between two operands (stored at R_a, R_b) and store the result in R_c
MOVE R_a, R_b	Move the data from address R_a to R_b
UPDATE_LOOP_COUNTER	Increment/Decrement the loop counter
JZ LOOP_COUNTER	Jump instruction when loop counter reaches zero
JNZ LOOP_COUNTER	Jump instruction when loop counter is non zero

data memory to general-purpose registers. *CM_ADD*, *CM_SUB* and *CM_MUL* instructions are used for performing complex addition, subtraction, and multiplication, respectively. Apart from these, we have a few instructions for updating loop counters and controlling the code flow.

BFU Type	Radix	Degree of Polynomial	Latency
Cooley-Tukey	Radix-2	2	13
		4	48
		8	136
		16	348
		32	844
		64	1980
		128	4540
		256	10236
Winograd	Radix-8	512	22780
		2	13
		4	48
		8	101
		16	1670
		32	1670
		64	1670
		128	19800
		256	19800
		512	19800

Table 3: Latency Required for FFT of Different Degree Polynomials.

5 EXPERIMENTAL RESULT

In this section, we have provided our experimental results for a variable-length FFT implementation of FALCON. We have used Verilog HDL for designing the complex number multiplier/adder/subtractor. The proposed architecture, including the complex number multiplier/adder/subtractor, has been implemented using the Vivado 2022.2 design suite. Testing and validation have been done using the ZCU104+ (xczu7ev-ffvc1156-2-e) FPGA board. All the results in Table 1, Table 3 and Table 4 has been obtained after routing and placement in Vivado design suite.

The overhead and frequency of our proposed architecture have been reported in Table 4. As mentioned in Section 3, a few works in the literature have implemented a hardware-software co-design [17] or purely hardware implementation for FALCON [9]. Although the work [9] proposes a hardware implementation for FALCON, the area overhead is very high, and the work does not report the overhead or latency for FFT module separately. All the other works [2, 10] have proposed ASIC implementation and are not comparable

Board	LUT	DSP	BRAM	FF	Frequency (Mhz)
ZCU104+	8396	9	9.5	2526	200

Table 4: Overhead and Frequency of Our Proposed Architecture.

to our work. The authors in [5] propose a hardware implementation for FFT but they have used approximate double-precision arithmetic operators. Also, the work in [5] is not flexible as it needs separate hardware for different types of BFUs whereas our design principle is reconfigurable to work as any type of BFUs through our instruction set architecture. We have shown in Table 3 that our proposed architecture for Winograd's radix-8 FFT outperforms the Cooley-Tukey FFT method in terms of the latency cycles needed while moving to higher degree polynomials (64, 512 etc.). Our FFT computation for polynomial degree 16, 32, 64 consumes same latency cycle as we have considered the length of the polynomial as 64 in all these three cases by appending zeros in the remaining positions. The same principle has been applied for polynomial lengths of 128, 256, 512 by applying FFT on 512-degree polynomial in all these three cases.

6 CONCLUSION

In this work, we have constructed a hardware-software co-design for FFT module of FALCON using Winograd's method. We have designed efficient low-latency complex number adder/subtractor/multiplier that accelerates the floating point operations during FFT. Moreover, Winograd's FFT saves the multiplication count, which reduces the latency for higher-degree polynomials during FFT computation compared to the Cooley-Tukey method. We can, therefore, infer that our suggested Winograd's FFT method is a more efficient option than the Cooley-Tukey method.

REFERENCES

- [1] Janice S Agnello. 1979. An Introduction to the Winograd Discrete Fourier Transform. (1979).
- [2] Ghada Alsuhli, Hani Saleh, Mahmoud Al-Qutayri, Baker Mohammad, and Thanos Stouraitis. 2024. Area and Power Efficient FFT/IFFT Processor for FALCON Post-Quantum Cryptography. *arXiv preprint arXiv:2401.10591* (2024).
- [3] Luke Beckwith, Duc Tri Nguyen, and Kris Gaj. 2023. Hardware Accelerators for Digital Signature Algorithms Dilithium and FALCON. *IEEE Design & Test* (2023).
- [4] AJAA Bekele. 2016. Cooley-tukey fft algorithms. *Advanced algorithms* (2016).
- [5] Nikhilesh Bhagat, Daniel Valencia, Amirhossein Alimohammad, and Fred Harris. 2018. High-throughput and compact FFT architectures using the Good-Thomas and Winograd algorithms. *IET Communications* 12, 8 (2018), 1011–1018.
- [6] Nikhilesh Vinayak Bhagat. 2016. *A novel high-throughput FFT architecture for wireless communication systems*. Ph.D. Dissertation. San Diego State University.
- [7] Richard E Blahut. 2010. *Fast algorithms for signal processing*. Cambridge University Press.
- [8] Xiangren Chen, Bohan Yang, Shouyi Yin, Shaojun Wei, and Leibo Liu. 2022. CFNTT: Scalable radix-2/4 NTT multiplication architecture with an efficient conflict-free memory mapping scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems* (2022), 94–126.
- [9] Emre Karabulut and Aydin Aysu. 2023. A hardware-software co-design for the discrete gaussian sampling of falcon digital signature. *Cryptology ePrint Archive* (2023).
- [10] Youngbeom Kim, Jingyo Song, and Seog Chung Seo. 2022. Accelerating falcon on ARMv8. *IEEE Access* 10 (2022), 44446–44460.
- [11] Kevin Kinningham, Philip Levis, Mark Anderson, Dan Boneh, Mark Horowitz, and Maurice Shih. 2019. Falcon—A flexible architecture for accelerating cryptography. In *2019 IEEE 16th International Conference on Mobile Ad Hoc and Sensor Systems (MASS)*. IEEE, 136–144.
- [12] Georg Land, Pascal Sasdrich, and Tim Güneysu. 2021. A hard crystal-implementing dilithium on reconfigurable hardware. In *International Conference*

- on Smart Card Research and Advanced Applications*. Springer, 210–230.
- [13] Suraj Mandal and Debapriya Basu Roy. 2024. KiD: A Hardware Design Framework Targeting Unified NTT Multiplication for CRYSTALS-Kyber and CRYSTALS-Dilithium on FPGA. In *2024 37th International Conference on VLSI Design and 2024 23rd International Conference on Embedded Systems (VLSID)*. IEEE, 455–460.
- [14] Stavros Paschalakis and Peter Lee. 2003. Double precision floating-point arithmetic on FPGAs. In *Proceedings. 2003 IEEE International Conference on Field-Programmable Technology (FPT)*(IEEE Cat. No. 03EX798). IEEE, 352–358.
- [15] Thomas Pornin. 2019. New efficient, constant-time implementations of falcon. *Cryptology ePrint Archive* (2019).
- [16] Mateusz Raciborski and Aleksandr Cariow. 2022. On the derivation of Winograd-type DFT algorithms for input sequences whose length is a power of two. *Electronics* 11, 9 (2022), 1342.
- [17] Michael Schmid, Dorian Amiet, Jan Wendler, Paul Zbinden, and Tao Wei. 2023. Falcon Takes Off-A Hardware Implementation of the Falcon Signature Scheme. *Cryptology ePrint Archive* (2023).
- [18] Shmuel Winograd. 1978. On computing the discrete Fourier transform. *Mathematics of computation* 32, 141 (1978), 175–199.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009